

Cheap Shop Catalog Store

Project Report

Author: Ali Ayhan Gunder - 2021400219

Course: Human Computer Interaction CMPE 496

Date: July 06, 2025

Contents

1	Executive Summary	2
2	Project Overview	2
2.1	Purpose	2
2.2	Key Features	2
3	Technical Architecture	3
3.1	Java Swing Application (CheapShop.java)	3
3.2	Web Application (cheapshop.html)	3
4	Technical Implementation	4
4.1	Java Swing: Data Flow and Logic	4
4.2	Web Application: Interactivity and Modals	4
5	User Interface Design	4
5.1	Java Swing UI	4
5.2	Web Application UI	5
6	Performance and Quality	5
6.1	Strengths	5
6.2	Areas for Improvement	5
7	Future Enhancements	6
7.1	Java Swing Application	6
7.2	Web Application	6
8	Conclusion	6

1. Executive Summary

This project presents two implementations of a "Cheap Shop" catalog store application: one developed in Java using the Swing GUI framework and another as a modern web application using HTML, CSS, and JavaScript. Both applications allow users to select catalog items, specify quantities, and view a summary invoice or cart before checkout. The project demonstrates principles of GUI design, event handling, and data management in both desktop and web environments.

2. Project Overview

2.1. Purpose

The "Cheap Shop" project aims to provide a functional catalog ordering system. The Java Swing version serves as a desktop application with detailed purchaser and item entry forms, while the web version offers a streamlined, interactive shopping experience. Both implementations showcase different approaches to user interface development and user interaction.

2.2. Key Features

- **Java Swing Application:**

- Purchaser information entry (name, phone, address, credit card, date, validation ID).
- Catalog item entry (item number, quantity, cost per item).
- Automatic calculation of total cost per item and running balance owing.
- "Next Catalog Item" functionality to add multiple items.
- "Trigger Invoice" to display a detailed summary and reset the application.
- Two distinct screens for initial purchaser entry + first item, and subsequent item entries.

- **Web Application (HTML/CSS/JavaScript):**

- List of predefined catalog items with prices.
- "Add to Cart" functionality for each item.
- Real-time update of cart item count.
- "View Cart" modal displaying selected items, quantities, and individual totals.
- Option to remove items from the cart.
- "Checkout" modal with order summary and total.
- Purchase confirmation and success message.
- Modern, responsive design with animations.

3. Technical Architecture

3.1. Java Swing Application (CheapShop.java)

The Java application is built using the `javax.swing` package.

- **JFrame:** The main application window.
- **JPanels:** Used for organizing UI components with `BorderLayout` and `GridBagLayout`.
- **JTextFields, JLabels, JSpinners, JButtons:** Standard Swing components for input and display.
- **ArrayList<CatalogItem>:** Stores selected catalog items.
- **CatalogItem Class:** A static inner class to encapsulate item data (number, quantity, cost, total).
- **Event Handling:** `ActionListener` for buttons, `KeyAdapter` for text fields, and `ChangeListener` for spinners manage user interactions.
- **Screen Management:** Methods `showPurchaserScreen()` and `showCatalogScreen()` dynamically update the `JFrame`'s content panel.

3.2. Web Application (cheapshop.html)

The web application is a single HTML file incorporating CSS for styling and JavaScript for interactivity.

- **HTML5 Structure:** Semantic HTML elements define the layout.
- **CSS3 Styling:**
 - Flexbox for layout management.
 - Gradients, shadows, and transitions for modern aesthetics and user feedback.
 - Responsive design for various screen sizes.
- **JavaScript Logic:**
 - `cart` array: Stores items added by the user.
 - `addToCart(itemName, price)`: Adds or updates an item in the cart.
 - `updateCartDisplay()`: Updates the cart count badge and enables/disables the checkout button.
 - `viewCart()`: Populates and displays the cart modal.
 - `removeFromCart(index)`: Removes an item from the cart array.
 - `checkout()`: Populates and displays the checkout confirmation modal.
 - `confirmPurchase()`: Clears the cart and displays a success message modal.
 - Modal management functions (`closeCart`, `closeCheckout`, `closeSuccessModal`).

4. Technical Implementation

4.1. Java Swing: Data Flow and Logic

- **Purchaser Information:** Entered once on the initial screen.
- **Catalog Item Entry:** Users input item number, quantity, and cost. `totalField` is automatically updated based on `quantitySpinner` and `costField` changes.
- **Balance Calculation:** `balanceOwing` is accumulated with each item added using `addCatalogItem()`.
- **Screen Transitions:** `addCatalogItem()` switches from the initial purchaser screen to the catalog-only screen for subsequent items. `triggerInvoice()` resets the application to the initial screen.
- **Invoice Generation:** `triggerInvoice()` compiles purchaser details and all added items into a formatted `JOptionPane` dialog.

4.2. Web Application: Interactivity and Modals

- **Dynamic Cart Updates:** The `cart` array is the central data structure. Changes to the cart trigger `updateCartDisplay()`.
- **Modal Dialogs:** Separate `div` elements with CSS are used to simulate modal dialogs for "View Cart" and "Checkout". Their `display` CSS property is toggled via JavaScript.
- **User Feedback:** Buttons provide visual "Added!" feedback upon item addition. Modals slide in with a `modalSlideIn` animation.
- **Event Delegation:** `onclick` attributes are used directly in HTML for simplicity, calling JavaScript functions.

5. User Interface Design

5.1. Java Swing UI

The design prioritizes clear labeling and logical grouping of fields.

- **Layout Managers:** `GridBagLayout` offers precise control over component placement within panels, ensuring fields align well. `BorderLayout` arranges the main panels.
- **Borders:** `TitledBorder` visually separates "Purchaser" and "Catalog Item" sections.
- **Input Components:** Standard `JTextField` for text input, `JSpinner` for quantity selection, enhancing usability for numerical input.
- **Button Placement:** "Next Catalog Item" and "Trigger Invoice" buttons are prominently placed.

- **Read-Only Fields:** `totalField` and `balanceField` are set to `editable(false)` to prevent manual input, indicating they are calculated values.

5.2. Web Application UI

The design focuses on modern aesthetics and an intuitive shopping flow.

- **Visual Hierarchy:** Clear headings and item cards.
- **Color Palette:** Use of gradients (`linear-gradient`) and complementary colors for a vibrant look.
- **Interactive Elements:** Buttons change style on hover and click, providing visual feedback.
- **Modals:** Well-styled pop-up windows for cart and checkout, ensuring user focus during critical actions.
- **Responsiveness:** Designed to adapt reasonably well to different screen sizes, characteristic of web applications.
- **Badges:** The `cart-count` badge provides immediate visual feedback on the number of items in the cart.

6. Performance and Quality

6.1. Strengths

- **Modularity (Java):** Clear separation of UI logic and data handling. The `CatalogItem` class encapsulates item properties effectively.
- **Robust Input Handling (Java):** Basic validation for empty fields and numeric parsing.
- **Interactive Experience (Web):** Smooth animations and immediate visual feedback enhance user engagement.
- **State Management (Web):** The `cart` array efficiently manages the current shopping state.
- **Code Clarity:** Both projects use descriptive variable names and clear function definitions, making them relatively easy to understand and maintain.

6.2. Areas for Improvement

- **Input Validation (Java):** More comprehensive validation (e.g., phone number format, postal code, credit card regex) could be implemented.
- **Error Handling (Java):** More specific error messages for different types of invalid input.

- **Persistence:** Neither application saves data across sessions (e.g., to a file or database).
- **Accessibility:** Enhancements for users with disabilities could be considered (e.g., keyboard navigation improvements in Java Swing, ARIA attributes in web app).
- **Scalability (Web):** For a larger e-commerce site, a more robust backend and framework (e.g., React, Angular, Vue) would be necessary for managing product data and complex user interactions.

7. Future Enhancements

7.1. Java Swing Application

- Integration with a database to store purchaser and item data.
- Implement an "undo" functionality for item additions.
- Add search/lookup functionality for catalog items.
- Generate a printable invoice document.
- Support for different payment methods.

7.2. Web Application

- Quantity adjustment directly in the cart modal.
- Product images and descriptions.
- User authentication and order history.
- Integration with a payment gateway.
- Backend for dynamic product listings and inventory management.

8. Conclusion

The "Cheap Shop Catalog Store" project successfully demonstrates the implementation of a simple e-commerce like application using two distinct technologies: Java Swing for a desktop environment and HTML/CSS/JavaScript for a web environment. Each version highlights the unique strengths and typical design patterns of its respective platform. The Java Swing application provides a robust form-based data entry system, while the web application focuses on a dynamic and visually appealing user experience. Both projects serve as valuable examples of fundamental GUI programming and user interaction design principles.